Call Me Maybe

Cours 4 — Le constrained decoding en profondeur (v2 — vraies fonctions)

---

1. Les vraies fonctions du projet

Ton fichier functions_definition.json contient 5 fonctions. Voici leur schema complet :

Nom de fonction	Parametres	Retour	Description
`fn_add_numbers`	a: number, b: number	number	Additionne deux nombres
`fn_greet`	name: string	string	Genere un message de salutation
`fn_reverse_string`	s: string	string	Inverse une chaine
`fn_get_square_root`	a: number	number	Racine carree
`fn_substitute_string_with_regex`	source_string, regex, replacement (3 x string)	string	Remplace via regex

fn_substitute_string_with_regex est la fonction la plus complexe : 3 parametres string. Ton programme doit gerer ce cas sans code specifique — la machine d'etats le gere automatiquement.

2. La machine d'etats

#	Etat	Deja genere	Tokens valides
1	START	(rien)	{ uniquement
2	OPEN_BRACE	{	" pour commencer la cle
3	KEY_NAME	{"	"name" ou "parameters" (tokens exacts)
4	COLON_NAME	{"name"	: puis espace
5	FUNC_NAME	{"name": "	Tokens qui continuent un nom de fonction valide

#	Etat	Deja genere	Tokens valides
6	COMMA	{"name": "fn_xxx"	, pour separer les deux cles
7	KEY_PARAMS	..., "	"parameters" (tokens exacts)
8	COLON_PARAMS	..."parameters"	: puis {
9	PARAM_KEY	..."parameters": {"	Noms de parametres de la fonction choisie
10	PARAM_VALUE	..."a":	Tokens du type attendu (number/string/bool)
11	PARAM_NEXT	...valeur generee	, si params restants, } si dernier param
12	END	...}}	Token EOS

3. get_valid_tokens() — implementation

```
from enum import Enum, auto
from dataclasses import dataclass, field
class JsonState(Enum):
    START=auto(); FUNC_NAME=auto(); PARAM_KEY=auto()
    PARAM_VALUE=auto(); PARAM_NEXT=auto(); END=auto()
@dataclass
class GenerationContext:
    state        : JsonState
    functions    : list[FunctionDef]
    func_tokens  : dict[str, list[int]]  # {nom: [id1,id2,...]}
    chosen_func  : FunctionDef | None = None
    tokens_so_far: list[int] = field(default_factory=list)
    params_done  : list[str] = field(default_factory=list)
    current_param: str | None = None
def get_valid_tokens(ctx: GenerationContext, vocab: dict[str,int]) -> list[int]:
    if ctx.state == JsonState.START:
        return [vocab['{']]
    if ctx.state == JsonState.FUNC_NAME:
        n = len(ctx.tokens_so_far)
        valid = set()
        for func_name, token_ids in ctx.func_tokens.items():
            if len(token_ids) > n and token_ids[:n] == ctx.tokens_so_far:
                valid.add(token_ids[n])
        if not valid:
            raise ValueError('Aucun token valide pour le nom de fonction')
        return list(valid)
    if ctx.state == JsonState.PARAM_VALUE:
        param_type = ctx.chosen_func.parameters[ctx.current_param].type
        if param_type == 'number' : return get_number_tokens(vocab)
        if param_type == 'string' : return get_string_tokens(vocab)
        if param_type == 'boolean': return [vocab['true'], vocab['false']]
    if ctx.state == JsonState.PARAM_NEXT:
        all_params = list(ctx.chosen_func.parameters.keys())
        remaining  = [p for p in all_params if p not in ctx.params_done]
        return [vocab[','']] if remaining else [vocab['}']]
```

4. Suivi token par token — exemple avec les vraies fonctions

Prompt : "Greet shrek" — le modele genere le nom de fonction :

Token genere	Fonctions possibles	Tokens valides suivants	Statut
`"fn"`	fn_add_numbers, fn_greet, fn_reverse_string, fn_get_square_root, fn_substitute_string_with_regex	"_add", "_greet", "_reverse", "_get", "_substitute"	5 fonctions possibles
`"_greet"`	fn_greet (seule restante)	"\""	Converge !
`"\""`	fn_greet (confirmee)	","	Fermer le nom, passer aux params

5. Generer les valeurs selon leur type

Type / Position	Tokens valides	Tokens invalides
`number — debut`	0-9, signe -	lettres, {, "
`number — milieu`	0-9, point ., e, E	lettres, {, "
`number — fin`	, ou }	lettres, chiffres
`string — debut`	" uniquement	tout le reste
`string — contenu`	tous les tokens sauf "	" (fermerait trop tot)
`string — fin`	, ou }	lettres, chiffres
`boolean`	true ou false (minuscules !)	True, False, 1, 0

6. Cas complexe : fn_substitute_string_with_regex

Cette fonction a 3 parametres string. La boucle PARAM_KEY -> PARAM_VALUE -> PARAM_NEXT tourne 3 fois. params_done permet de savoir quand mettre , ou } :

```
# Apres avoir genere source_string:
params_done = ['source_string']
remaining   = ['regex', 'replacement']  # -> token valide : ,
# Apres avoir genere regex:
params_done = ['source_string', 'regex']
remaining   = ['replacement']           # -> token valide : ,
# Apres avoir genere replacement:
params_done = ['source_string', 'regex', 'replacement']
remaining   = []                        # -> token valide : }
```

Exercice

Q1. Fonctions dispo : fn_greet et fn_get_square_root. Le modele a genere {"name": "fn_g. Quels tokens sont valides ?

Q2. Le modele genere le parametre a de type number, il a produit 2. Quels tokens sont valides ensuite ? Donne 4 valides et 2 invalides.

Q3. fn_substitute_string_with_regex : params a, b, c. Le modele finit b. Quels tokens valides ? Pourquoi different de la fin de c ?

Reponses

Q 1	Tokens valides : "reet" (pour fn_greet) et "et_" ou "_get" (pour fn_get_square_root). Pas les noms complets — on cherche la suite token par token.
Q 2	Valides : chiffres 0-9, point . (decimal), e/E (scientifique), , ou } (fin). Invalides : espace, lettre b, guillemets, accolades ouvrantes.
Q 3	Apres b : token valide = , (il reste c). Apres c : token valide = } (c'est le dernier). Le programme compare params_done avec la liste complete de la fonction.

A retenir

- 5 fonctions reelles : fn_add_numbers, fn_greet, fn_reverse_string, fn_get_square_root, fn_substitute_string_with_regex.
- La machine d'etats a 12 etats. Les etats PARAM_* forment une boucle pour chaque parametre.
- get_valid_tokens() consulte l'etat courant, la fonction choisie et params_done.
- params_done permet de savoir si on met , ou } apres chaque valeur.